

Fase 2 - Maven, .m2 e pom.xml

Esse material cobre o que é o Maven, como ele gerencia dependências, o que é o .m2 e como ler um pom.xml. Sempre que possível, abra o pom.xml do projeto que você criou na Fase 0 e vá comparando com os exemplos abaixo.

1. O que é o Maven e que problema ele resolve

Antes do Maven, se você quisesse usar uma biblioteca (por exemplo, uma lib pra trabalhar com JSON), você tinha que:

1. Ir no site da biblioteca
2. Baixar o .jar manualmente
3. Colocar esse .jar dentro do seu projeto
4. Configurar o classpath pra apontar pra ele
5. Torcer pra ninguém mais no time esquecer de fazer isso também

O Maven automatiza tudo isso. Você só declara “eu preciso da biblioteca X, versão Y” num arquivo de configuração (o pom.xml), e o Maven:

- Baixa essa biblioteca automaticamente de um repositório público
- Baixa qualquer outra biblioteca que ela precise (dependência transitiva)
- Guarda tudo numa pasta local pra não precisar baixar de novo depois
- Sabe como compilar, testar e empacotar o projeto com um comando só

Além de gerenciar dependências, o Maven também define um **ciclo de build** — uma sequência padronizada de passos (compilar, testar, empacotar) que funciona igual em qualquer projeto Maven, seja ele qual for.

2. O que é o .m2 (repositório local)

Quando o Maven baixa uma biblioteca, ele não baixa toda vez que você compila. Ele guarda um cache numa pasta no seu computador chamada .m2 (geralmente em C:\Users\SEU_USUARIO\.m2 no Windows, ou ~/.m2 no Mac/Linux).

Dentro dela fica uma pasta chamada repository, organizada mais ou menos assim:

```
.m2
  repository
    org
      springframework
        boot
          spring-boot-starter-web
            3.2.0
              spring-boot-starter-web-3.2.0.jar
```

```
com
  fasterxml
    jackson-core
    ...
```

Ou seja: a primeira vez que você compila um projeto que usa o Spring Boot, o Maven baixa um monte de coisa e pode demorar. Da segunda vez em diante, ele já acha tudo pronto no .m2 e o build fica bem mais rápido.

Ponto prático: se um dia você tiver um erro estranho de dependência “corrompida” ou “incompleta”, uma solução comum é apagar a pasta específica daquela biblioteca dentro do .m2 e mandar o Maven baixar de novo. Isso não apaga nada do seu código — só limpa o cache de bibliotecas baixadas.

3. Anatomia do pom.xml

pom.xml significa “Project Object Model”. É o arquivo de configuração principal de um projeto Maven, sempre na raiz do projeto. Vamos ver as partes principais:

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.empresa</groupId>
  <artifactId>estudos-java</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
      <version>3.2.0</version>
    </dependency>
  </dependencies>
</project>
```

Explicando cada parte:

- **groupId:** identifica quem é o “dono” do projeto (geralmente o domínio da empresa ao contrário, tipo com.empresa). No caso de bibliotecas de terceiros, é quem publicou aquela lib.
- **artifactId:** o nome do projeto/biblioteca em si.
- **version:** a versão daquele projeto/biblioteca específica.

Juntos, groupId + artifactId + version formam o que chamamos de **coordenadas Maven** — é como o Maven identifica exatamente qual biblioteca (e qual versão dela) você quer usar. É também esse trio que forma o caminho de pastas dentro do .m2 que vimos no tópico 2.

- **properties:** configurações gerais do projeto, como qual versão do Java usar pra compilar.
- **dependencies:** a lista de bibliotecas que o seu projeto precisa. Cada <dependency> dentro dela é uma biblioteca, identificada pelas mesmas coordenadas (groupId, artifactId, version).

4. Dependência transitiva

Quando você declara uma dependência, ela pode precisar de outras dependências pra funcionar — e o Maven baixa essas outras automaticamente, sem você precisar declarar uma por uma. Isso se chama **dependência transitiva**.

Exemplo: se você adiciona só isso no pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <version>3.2.0</version>
</dependency>
```

O Maven baixa, sozinho, dezenas de outras bibliotecas que o Spring Web precisa por baixo dos panos (Tomcat embutido, Jackson pra JSON, etc). Você não vê isso no seu pom.xml, mas dá pra visualizar essa “árvore de dependências” no próprio IntelliJ (tem uma aba/painel chamado “Maven” que mostra isso).

5. Scope das dependências

Cada dependência pode ter um scope (escopo), que define **onde** ela fica disponível. Os mais comuns:

- **compile** (padrão, quando você não escreve nada): disponível em todo o projeto — compilação, testes e quando a aplicação roda de verdade.
- **test:** disponível só durante os testes, não vai pro artefato final que roda em produção. Exemplo típico:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

- **provided:** o código precisa dela pra compilar, mas ela já vai estar disponível no ambiente onde a aplicação roda (menos comum no dia a dia com Spring Boot, mais comum em outros tipos de projeto Java).

6. Comandos básicos do Maven

Você pode rodar esses comandos pelo terminal (na pasta do projeto) ou direto pela aba “Maven” do IntelliJ, clicando duas vezes no comando.

- **mvn clean** — Apaga a pasta target (o que tinha sido compilado antes). Bom pra garantir que você vai compilar do zero, sem sobra de build antigo.

- **mvn compile** — Compila o código-fonte (transforma .java em .class).
- **mvn test** — Roda os testes automatizados do projeto.
- **mvn package** — Compila, roda os testes e empacota tudo num arquivo .jar (ou .war), a versão “pronta pra rodar” da sua aplicação. Esse arquivo fica dentro da pasta target.
- **mvn install** — Faz tudo que o package faz, e além disso copia esse pacote gerado pro seu .m2 local — útil quando outro projeto seu, no mesmo computador, depende desse aqui.

Combinação comum no dia a dia: `mvn clean install` (limpa tudo, recompila do zero, testa e empacota)

Exercício 2.1 Abra o `pom.xml` do projeto que você criou na Fase 0 e responda: 1. Qual é o `groupId` e o `artifactId` do seu projeto? 2. Qual versão do Java está configurada nas `properties`? 3. Existe alguma `dependency` declarada? Se sim, qual(is)?

Exercício 2.2 No seu projeto (o mesmo da Fase 0), adicione essa dependência dentro da tag `<dependencies>` do `pom.xml`:

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.14.0</version>
</dependency>
```

Depois de salvar o `pom.xml`, o IntelliJ geralmente pergunta se você quer “importar as mudanças do Maven” (um íconezinho aparece no canto). Aceite.

Depois de importado, abra a pasta do seu .m2 (repository) no explorador de arquivos do seu sistema operacional e ache a pasta correspondente a essa biblioteca (organização: `org/apache/commons/commons-lang3/3.14.0`). Confirme que o .jar dela realmente foi baixado ali.

Exercício 2.3 Ainda usando a dependência do exercício anterior, use uma classe dela no seu código pra confirmar que a dependência está funcionando de verdade (não só baixada, mas utilizável):

```
import org.apache.commons.lang3.StringUtils;

public class Main {
    public static void main(String[] args) {
        System.out.println(StringUtils.isBlank("  ")); // true
        System.out.println(StringUtils.isBlank("texto")); // false
    }
}
```

Rode e confirme a saída. Depois, no terminal (dentro da pasta do projeto), rode o comando:

```
mvn clean package
```

Observe a saída no terminal e responda: quanto tempo levou o build? Quais fases apareceram no log (compile, test, package)? Onde ficou o arquivo .jar

gerado?